Step 4 — La loss: classificazione binaria sul margine fra le due lettere

Obiettivo

Fissare la funzione obiettivo per il compito unico. La regola resta quella del piano generale: **si addestra esattamente ciò che si legge in inferenza**, cioè i due logit delle lettere nella posizione dopo `Answer:`. Con una sola domanda binaria la loss si semplifica: non c'è KL di replay, non ci sono domande a più di due opzioni.

1. Le lettere sono una testa binaria

Con `z_vero`, `z_falso` i logit delle due lettere e `T` la temperatura del modello:

```
s = (z_vero − z_falso) / T          margine
P(supportato) = σ(s)
```

I due logit vengono dalle righe della testa del vocabolario per le opzioni (8 decision model e i backbone) o dalla testa propria del modello (pplx-decider-flash, Clef-Flash). T è la temperatura degli autori (es. KnowLine 1, già nei pesi; StartLux-9B 1,3396; pplx 1,554). L'ordine delle opzioni cambia da modello a modello (KnowLine: A = sì; d3-flash, Jeffan, pplx: A = no): si usa sempre l'opzione "supportato".

Siccome `z_k = w_k · h` (riga `k` della testa del vocabolario per lo stato nascosto `h`), `s = u · h` con `u = (w_vero − w_falso) / T`. È una **regressione logistica su `h`** con pesi già inizializzati: la stessa struttura della testa `Linear(1024→2)` di FactCG/MiniCheck (2 logit con softmax = 1 margine con sigmoide). Discussione completa in 00a §4.

2. Cross-entropy binaria (default)

```
L_CE = −[ y · ln σ(s) + (1 − y) · ln(1 − σ(s)) ]
dL/ds = σ(s) − y
```

Esempio con il bias attuale di `support_simple` (dice "sì" troppo facilmente). Coppia **non** supportata (y = 0), il modello di partenza dà s = 2,2:

	Valore
P(supportato) = σ(2,2)	0,90
L_CE = −ln(1 − 0,90)	2,30
Gradiente su s: σ(s) − y	+0,90 (spinge s verso il basso con forza)

Dopo qualche centinaio di passi, s = −0,5 → P = 0,38, L_CE = 0,48, gradiente +0,38. È esattamente il meccanismo che deve riportare la soglia ottimale da 0,90–0,95 verso 0,5.

È la loss di FactCG (`CE / ln 2`, solo un fattore di scala) e di MiniCheck.

3. Varianti contro il rumore delle etichette

I dati generati e ANLI difficile (02a §3) contengono etichette sbagliate. Con la CE, un esempio sbagliato ma "sicuro" spinge al massimo (gradiente vicino a ±1).

Loss	Formula	Gradiente su s per y = 1, σ(s) = 0,01	Quando
CE	−ln σ(s)	−0,99	default
Brier (2 opzioni)	2 (σ(s) − y)²	≈ **−0,04** (calcolo sotto)	molti dati generati
CE con label smoothing ε	y' = y(1−ε) + ε/2	−(0,975 − 0,01) = −0,965 (ε = 0,05)	rumore moderato; tiene le probabilità lontane da 0 e 1

Brier: `L = (σ(s) − y)² + ((1 − σ(s)) − (1 − y))² = 2(σ(s) − y)²`, gradiente su s = `4 (σ(s) − y) σ(s)(1 − σ(s))`. Con σ = 0,01, y = 1: 4 · (−0,99) · 0,0099 ≈ −0,04.

Scelta iniziale: **CE**. Esperimento P2 (07): Brier; P-LS: smoothing ε = 0,05 solo sulle fonti generate e su ANLI difficile.

4. Ancoraggio opzionale al modello di partenza

```
L = L_CE + β · KL(p_0 ‖ p),      p_0 = P(supportato) del modello di partenza (precalcolata)
KL binaria = p_0 ln(p_0/p) + (1−p_0) ln((1−p_0)/(1−p))
```

Esempio: p_0 = 0,80, p = 0,60 → KL = 0,8 · ln(1,333) + 0,2 · ln(0,5) = 0,230 − 0,139 = **0,092**.

β ∈ {0,05; 0,1}, solo nell'esperimento P-KL (02c spiega perché non è il default: conserverebbe anche il bias verso "sì").

5. Loss contrastiva su coppie minime (opzionale)

Le famiglie di dati di questo piano producono **coppie** per costruzione: C2D (documento intero contro documento senza una frase, stesso claim), CG2C (documento contro documento senza una relazione), `remove_evidence`, `contradict_claim`, VitaminC.

```
L_pair = −ln σ(s_pos − s_neg)
```

Esempio: s_pos = 1,2, s_neg = 0,9 → differenza 0,3 → L_pair = 0,55; con differenza 3 → 0,05. Spinge il modello a guardare proprio la frase o la parola che cambia.

Peso μ ∈ {0,1; 0,3}, sul 20–30% degli esempi che hanno una coppia; i due membri nello stesso micro-batch. Esperimenti X1–X2.

6. La loss totale

```
L = media[ CE (o Brier, o CE con smoothing) ]
  + β · media[ KL(p_0 ‖ p) ]              (opzionale, β = 0 di default)
  + μ · media sulle coppie[ L_pair ]       (opzionale, μ = 0 di default)
```

7. Codice di riferimento (per `finetune/train.py`)

```python
import torch
import torch.nn.functional as F

def margin(logits_at_answer, true_id, false_id, temperature):
    """logits_at_answer [B, V] -> margine s [B] = (z_vero - z_falso) / T."""
    return (logits_at_answer[:, true_id] - logits_at_answer[:, false_id]) / temperature

def binary_loss(s, y, kind="ce", smoothing=0.0):
    """s: [B] margine; y: [B] in {0, 1} (float)."""
    if kind == "ce":
        y = y * (1 - smoothing) + smoothing / 2
        return F.binary_cross_entropy_with_logits(s, y)
    if kind == "brier":
        return (2 * (torch.sigmoid(s) - y) ** 2).mean()
    raise ValueError(kind)

def anchor_kl(s, p0, eps=1e-6):
    p = torch.sigmoid(s).clamp(eps, 1 - eps)
    p0 = p0.clamp(eps, 1 - eps)
    return (p0 * (p0 / p).log() + (1 - p0) * ((1 - p0) / (1 - p)).log()).mean()

def pair_loss(s_pos, s_neg):
    return F.softplus(-(s_pos - s_neg)).mean()      # = -log sigmoid(s_pos - s_neg)
```

Per non calcolare i logit di tutto il vocabolario su tutta la sequenza: prendere lo stato nascosto finale in `answer_pos` e moltiplicarlo **solo** per le due righe `w_vero`, `w_falso` della testa (un prodotto `[B, d] × [d, 2]`). Per pplx e Clef si chiama solo la loro testa. Il margine così calcolato deve dare la stessa P(supportato) dell'adapter del benchmark. Il test di 05 §6 lo verifica.

Criteri per passare allo step successivo

- Funzioni testate sui casi numerici di questo file.
- Verificato che il margine calcolato con le due righe coincide con quello del forward completo (differenza < 1e-3).